

CHAPTER 2

Database System Architecture

- 2.1 Introduction
- 2.2 The Three Levels of the Architecture
- 2.3 The External Level
- 2.4 The Conceptual Level
- 2.5 The Internal Level
- 2.6 Mappings
- 2.7 The Database Administrator
- 2.8 The Database Management System
- 2.9 Data Communications
- 2.10 Client/Server Architecture
- 2.11 Utilities
- 2.12 Distributed Processing
- 2.13 Summary
- Exercises
- References and Bibliography

2.1 INTRODUCTION

We are now in a position to present an architecture for a database system. Our aim in presenting this architecture is to provide a framework on which subsequent chapters can build. Such a framework is useful for describing general database concepts and for explaining the structure of specific database systems—but we do not claim that every system can neatly be matched to this particular framework, nor do we mean to suggest that

this particular architecture provides the only possible framework. In particular, "small" systems (see Chapter 1) will probably not support all aspects of the architecture. However, the architecture does seem to fit most systems reasonably well; moreover, it is basically identical to the architecture proposed by the ANSI/SPARC Study Group on Data Base Management Systems (the so-called ANSI/SPARC architecture—see references [2.1] and [2.2]). We choose not to follow the ANSI/SPARC terminology in every detail, however.

Caveat: This chapter resembles Chapter 1 inasmuch as, while an understanding of the material it contains is essential to a full appreciation of the structure and capabilities of a modern database system, it is again somewhat abstract and dry. As with Chapter 1, therefore, you might prefer just to give the material a "once over lightly" reading for now and come back to it later as it becomes more directly relevant to the topics at hand.

2.2 THE THREE LEVELS OF THE ARCHITECTURE

The ANSI/SPARC architecture is divided into three levels, usually referred to as the internal level, the external level, and the conceptual level (see Fig. 2.1), though other names are also used. Broadly speaking:

- The internal level (also known as the *storage level*) is the one closest to physical storage—that is, it is the one concerned with the way the data is stored inside the system.
- The external level (also known as the *user logical level*) is the one closest to the users—that is, it is the one concerned with the way the data is seen by individual users.
- The conceptual level (also known as the *community logical level*, or sometimes just the *logical level*, unqualified) is a level of indirection between the other two.

Observe that the external level is concerned with *individual* user perceptions, while the conceptual level is concerned with a *community* user perception. As we saw in Chapter 1, most users will not be interested in the total database, but only in some restricted

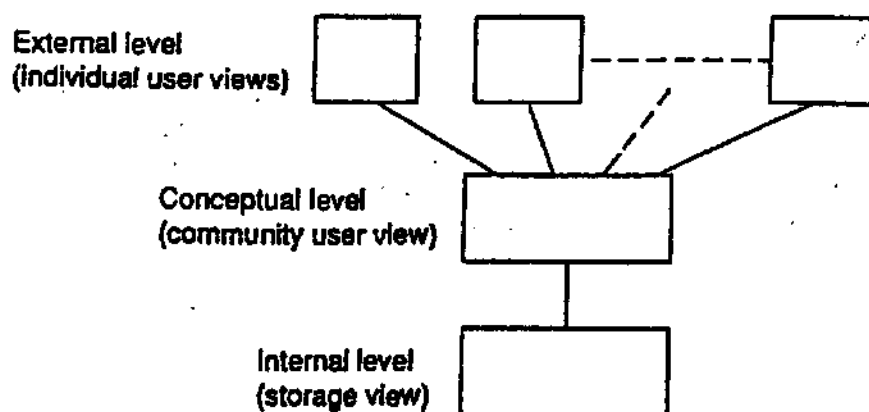


Fig. 2.1 The three levels of the architecture

portion of it; thus, there will be many distinct "external views," each consisting of a more or less abstract representation of some portion of the total database, and there will be precisely one "conceptual view," consisting of a similarly abstract representation of the database in its entirety. And then there will be precisely one "internal view," representing the database as stored internally. Note that (to use the terminology of Chapter 1) the external and conceptual levels are both *model* levels, while the internal level is an *implementation* level; in other words, the external and conceptual levels are defined in terms of user-oriented constructs such as records and fields, while the internal level is defined in terms of machine-oriented constructs such as bits and bytes.

An example will help to make these ideas clearer. Fig. 2.2 shows the conceptual view, the corresponding internal view, and two corresponding external views (one for a PL/I user and one for a COBOL user¹), all for a simple personnel database. Of course, the example is completely hypothetical—it is not intended to resemble any real system—and many irrelevant details have deliberately been omitted. *Explanation:*

1. At the conceptual level, the database contains information concerning an entity type called EMPLOYEE. Each individual employee has an EMPLOYEE_NUMBER (six characters), a DEPARTMENT_NUMBER (four characters), and a SALARY (five decimal digits).
2. At the internal level, employees are represented by a stored record type called STORED_EMP, 20 bytes long. STORED_EMP contains four stored fields: a 6-byte prefix (presumably containing control information such as codes, flags, or pointers), and three data fields corresponding to the three properties of employees. In addition, STORED_EMP records are indexed on the EMP# field by an index called EMPX, whose definition is not shown.

External (PL/I)	External (COBOL)
DCL 1 EMPP, 2 EMP# CHAR(6), 2 SAL FIXED BIN(31);	01 EMPC. 02 EMPNO PIC X(6). 02 DEPTNO PIC X(4).
Conceptual	
EMPLOYEE	
EMPLOYEE_NUMBER	CHARACTER(6)
DEPARTMENT_NUMBER	CHARACTER(4)
SALARY	DECIMAL(5)
Internal	
STORED_EMP	BYTES=20
PREFIX	BYTES=6,OFFSET=0
EMP#	BYTES=6,OFFSET=6,INDEX=EMPX
DEPT#	BYTES=4,OFFSET=12
PAY	BYTES=4,ALIGN=FULLWORD,OFFSET=16

Fig. 2.2 An example of the three levels

¹ We apologize for using such ancient languages as the basis for this example, but the fact is that PL/I and COBOL are both still widely used in commercial installations.

3. The PL/I user has an external view of the database in which each employee is represented by a PL/I record containing two fields (department numbers are of no interest to this user and have therefore been omitted). The record type is defined by an ordinary PL/I structure declaration in accordance with normal PL/I rules.
4. Similarly, the COBOL user has an external view in which each employee is represented by a COBOL record containing, again, two fields (this time, salaries have been omitted). The record type is defined by an ordinary COBOL record description in accordance with normal COBOL rules.

Notice that corresponding data items can have different names at different points in the foregoing scheme. For example, the employee number is called EMP# in the PL/I external view, EMPNO in the COBOL external view, EMPLOYEE_NUMBER in the conceptual view, and EMP# again in the internal view. Of course, the system must be aware of the correspondences; for example, it must be told that the COBOL field EMPNO is derived from the conceptual field EMPLOYEE_NUMBER, which in turn is derived from the stored field EMP# at the internal level. Such correspondences, or mappings, are not explicitly shown in Fig. 2.2; see Section 2.6 for further discussion.

Now, it makes little difference for the purposes of the present chapter whether the system under consideration is relational or otherwise. However, it might be helpful to indicate briefly how the three levels of the architecture are typically realized in a relational system specifically:

- First, the conceptual level in such a system will definitely be relational, in the sense that the objects visible at that level will be relational tables and the operators will be relational operators (including in particular the *restrict* and *project* operators discussed briefly in Chapter 1).
- Second, a given external view will typically be relational too, or something very close to it; for example, the PL/I and COBOL record declarations of Fig. 2.2 might loosely be regarded as PL/I and COBOL analogs of the declaration of a relational table in a relational system. *Note:* We should mention in passing that the term *external view* (usually abbreviated to just *view*) unfortunately has a rather specific meaning in relational contexts that is *not* identical to the meaning assigned to it in this chapter. See Chapters 3 and (especially) 10 for an explanation and discussion of the relational meaning.
- Third, the internal level will *not* be relational, because the objects at that level will not be just (stored) relational tables—instead, they will be the same kinds of objects found at the internal level of any other kind of system (stored records, pointers, indexes, hashes, etc.). In fact, the relational model as such has nothing whatsoever to say about the internal level; it is, to repeat from Chapter 1, concerned with how the database looks to the *user*.

We now proceed to discuss the three levels of the architecture in considerably more detail, starting with the external level. Throughout our discussions we will be making repeated references to Fig. 2.3, which shows the major components of the architecture and their interrelationships.

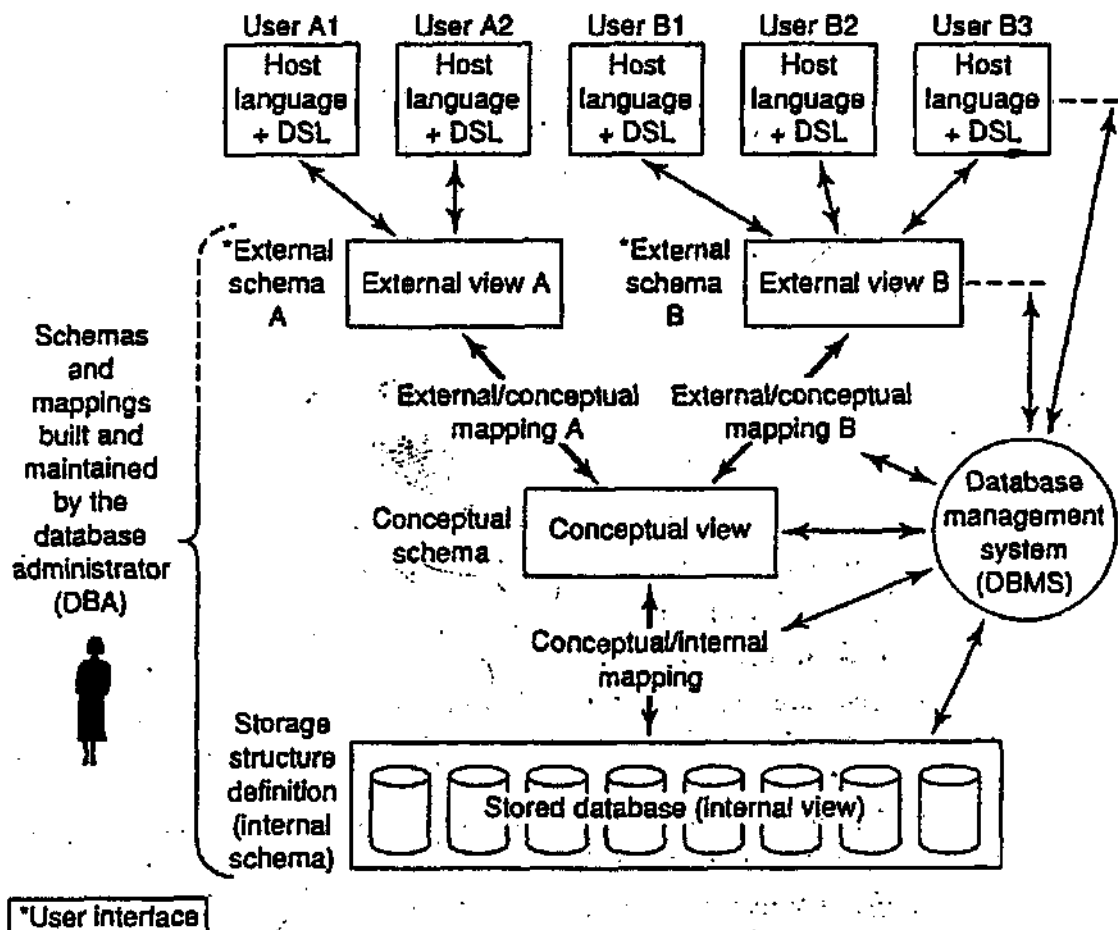


Fig. 2.3 Detailed system architecture

2.3 THE EXTERNAL LEVEL

The external level is the individual user level. As explained in Chapter 1, a given user can be an application programmer or an end user of any degree of sophistication. (The DBA is an important special case; unlike other users, however, the DBA will need to be interested in the conceptual and internal levels also. See the next two sections.)

Each user has a language at his or her disposal:

- For the application programmer, that language will be either a conventional programming language (e.g., Java, C++, or PL/I) or perhaps a proprietary language that is specific to the system in question. Such proprietary languages are sometimes called *fourth-generation* languages or 4GLs, on the (fuzzy!) grounds that (a) machine code, assembler language, and languages such as Java or C++ or PL/I can be regarded as three earlier language "generations," and (b) the proprietary languages represent the same kind of improvement over third-generation languages (3GLs) as those languages did over assembler language and assembler language did over machine code.

- For the end user, the language will be either a query language (probably SQL) or some special-purpose language, perhaps forms- or menu-driven, tailored to that user's requirements and supported by some online application as explained in Chapter 1.

For our purposes, the important thing about all such languages is that they will include a data sublanguage—that is, a subset of the total language that is concerned specifically with database objects and operations. The data sublanguage (abbreviated DSL in Fig. 2.3) is said to be embedded within the corresponding host language. The host language is responsible for providing various nondatabase facilities, such as local variables, computational operations, branching logic, and so on. A given system might support any number of host languages and any number of data sublanguages; however, one particular data sublanguage that is supported by almost all current systems is the language SQL, discussed briefly in Chapter 1. Most such systems allow SQL to be used both *interactively* as a stand-alone query language and also *embedded* in other languages such as Java or C++ or PL/I (see Chapter 4 for further discussion).

Now, although it is convenient for architectural purposes to distinguish between the data sublanguage and its containing host language, the two might in fact *not* be distinct as far as the user is concerned; indeed, it is probably preferable from the user's perspective if they are not. If they are not distinct, or if they can be distinguished only with difficulty, we say they are tightly coupled (and the combination is called a *database programming language*²). If they are clearly and easily separable, we say they are loosely coupled. Some commercial systems—including in particular certain SQL products, such as Oracle—support tight coupling, but not all do (tight coupling provides a more uniform set of facilities for the user but obviously involves more effort on the part of the system implementers, a fact that presumably accounts for the *status quo*).

In principle, any given data sublanguage is really a combination of at least two subordinate languages—a data definition language (DDL), which supports the *definition* or “declaration” of database objects, and a data manipulation language (DML), which supports the *processing* or “manipulation” of such objects.³ For example, consider the PL/I user of Fig. 2.2 in Section 2.2. The data sublanguage for that user consists of those PL/I features that are used to communicate with the DBMS:

- The DDL portion consists of those declarative constructs of PL/I that are needed to declare database objects—the DECLARE (DCL) statement itself, certain PL/I data types, and possibly special extensions to PL/I to deal with new kinds of objects not supported by existing PL/I.
- The DML portion consists of those executable statements of PL/I that transfer information into and out of the database—again, possibly including special new statements.

² The language Tutorial D that we will be using in later chapters as a basis for examples—see the remarks on this topic in the preface to this book—is a database programming language in this sense.

³ This rather inappropriate use of the term *manipulation* has become sanctioned by usage.

(In the interest of accuracy, we should make it clear that PL/I does not in fact include any specific database features at the time of writing. The "DML" statements in particular are typically just PL/I CALL statements that invoke the DBMS—though those calls might be syntactically disguised in some way to make them a little more user-friendly; see the discussion of *embedded SQL* in Chapter 4.)

To return to the architecture: We have already indicated that an individual user will generally be interested only in some portion of the total database; moreover, that user's view of that portion will generally be somewhat abstract when compared with the way the data is physically stored. The ANSI/SPARC term for an individual user's view is an *external view*. An external view is thus the content of the database as seen by some particular user; to that user, in other words, the external view *is* the database. For example, a user from the Personnel Department might regard the database as a collection of department and employee record occurrences, and might be quite unaware of the supplier and part record occurrences seen by users in the Purchasing Department.

In general, then, an external view consists of many occurrences of many types of external record (*not necessarily the same thing as a stored record*).⁴ The user's data sublanguage is thus defined in terms of external records; for example, a DML *retrieve* operation will retrieve external record occurrences, not stored record occurrences. (Incidentally, we can now see that the term *logical record* used a couple of times in Chapter 1 actually referred to an external record. From this point forward, in fact, we will generally avoid the term *logical record*.)

Each external view is defined by means of an external schema, which consists basically of definitions of each of the various external record types in that external view (again, refer back to Fig. 2.2 for a couple of simple examples). The external schema is written using the DDL portion of the user's data sublanguage. (That DDL is therefore sometimes referred to as an external DDL.) For example, the employee external record type might be defined as a six-character employee number field plus a five-digit (decimal) salary field, and so on. In addition, there must be a definition of the *mapping* between the external schema and the underlying *conceptual* schema (see the next section). We will consider that mapping later, in Section 2.6.

2.4 THE CONCEPTUAL LEVEL

The conceptual view is a representation of the entire information content of the database, again (as with an external view) in a form that is somewhat abstract in comparison with the way in which the data is physically stored. It will also be quite different, in general, from the way in which the data is viewed by any particular user. Broadly speaking, the

⁴ We are assuming here that all information is represented at the external level in the form of records specifically. However, some systems allow information to be represented in other ways instead of or as well as records. For a system using such alternative methods, the definitions and explanations given in this section will require suitable modification. Analogous remarks apply to the conceptual and internal levels also. Detailed consideration of such matters is beyond the scope of this early part of the book: see Chapters 14 (especially the "References and Bibliography" section) and 25 for further discussion. See also—in connection with the internal level in particular—Appendix A.

conceptual view is intended to be a view of the data "as it really is," rather than as users are forced to see it by the limitations of (for example) the particular language or hardware they might be using.

The conceptual view consists of many occurrences of many types of conceptual record. For example, it might consist of a collection of department record occurrences, plus a collection of employee record occurrences, plus a collection of supplier record occurrences, plus a collection of part record occurrences, and so on. A conceptual record is not necessarily the same as either an external record, on the one hand, or a stored record, on the other.

The conceptual view is defined by means of the conceptual schema, which includes definitions of each of the various conceptual record types (again, refer to Fig. 2.2 for a simple example). The conceptual schema is written using another data definition language, the conceptual DDL. If physical data independence is to be achieved, then those conceptual DDL definitions must not involve any considerations of physical representation or access technique at all—they must be definitions of information content *only*. Thus, there must be no reference in the conceptual schema to stored field representation, stored record sequence, indexes, hashing schemes, pointers, or any other storage and access details. If the conceptual schema is made truly data-independent in this way, then the external schemas, which are defined in terms of the conceptual schema (see Section 2.6), will *a fortiori* be data-independent too.

The conceptual view, then, is a view of the total database content, and the conceptual schema is a definition of that view. However, it would be misleading to suggest that the conceptual schema is nothing more than a set of definitions much like the simple record definitions found in (e.g.) a COBOL program today. The definitions in the conceptual schema are intended to include a great many additional features, such as the security and integrity constraints mentioned in Chapter 1. Some authorities would go as far as to suggest that the ultimate objective of the conceptual schema is to describe the complete enterprise—not just its data *per se*, but also how that data is used: how it flows from point to point within the enterprise, what it is used for at each point, what audit or other controls are to be applied at each point, and so on [2.3]. It must be emphasized, however, that no system today actually supports a conceptual schema of anything approaching this degree of comprehensiveness;⁵ in most existing systems, the "conceptual schema" is little more than a simple union of all of the individual external schemas, plus certain security and integrity constraints. But it is certainly possible that systems of the future will be much more sophisticated in their support of the conceptual level.

2.5 THE INTERNAL LEVEL

The third level of the architecture is the internal level. The internal view is a low-level representation of the entire database; it consists of many occurrences of many types of internal record. *Internal record* is the ANSI/SPARC term for the construct that we have

⁵ Some might argue that the so-called *business rule* systems come close (see Chapters 9 and 14).

been calling a *stored* record (and we will continue to use this latter term). The internal view is thus still at one remove from the physical level, since it does not deal in terms of *physical* records—also called blocks or pages—nor with any device-specific considerations such as cylinder or track sizes. In other words, the internal view effectively assumes an unbounded linear address space; details of how that address space is mapped to physical storage are highly system-specific and are deliberately omitted from the general architecture. *Note:* In case you are not familiar with the term, we should explain that the block, or page, is the unit of I/O—that is, it is the amount of data transferred between secondary storage and main memory in a single I/O operation. Typical page sizes can be anywhere from 1KB or less to 64KB or so, where (as we will see later) 1KB = one kilobyte = 1024 bytes.

The internal view is described by means of the internal schema, which not only defines the various stored record types but also specifies what indexes exist, how stored fields are represented, what physical sequence the stored records are in, and so on (once again, see Fig. 2.2 for a simple example; see also Appendix D, online). The internal schema is written using yet another data definition language—the internal DDL.

Note: In what follows, we will tend to use the more intuitive terms *stored database* and *stored database definition* in place of *internal view* and *internal schema*, respectively. Also, we observe that, in certain exceptional situations, application programs—in particular, those of a utility nature (see Section 2.11)—might be permitted to operate directly at the internal level rather than at the external level. Needless to say, the practice is not recommended; it represents a security risk (since the security constraints are bypassed) and an integrity risk (since the integrity constraints are bypassed likewise), and the program will be data-dependent to boot; but sometimes it might be the only way to obtain the required functionality or performance—just as an application programmer might occasionally have to descend to assembler language in order to satisfy certain functionality or performance objectives in a programming language system.

2.6 MAPPINGS

In addition to the three levels *per se*, the architecture of Fig. 2.3 involves certain mappings—one conceptual/internal mapping and several external/conceptual mappings, in general:

- The *conceptual/internal* mapping defines the correspondence between the conceptual view and the stored database; it specifies how conceptual records and fields are represented at the internal level. If the structure of the stored database is changed—that is, if a change is made to the stored database definition—then the conceptual/internal mapping must be changed accordingly, so that the conceptual schema can remain invariant. (It is the responsibility of the DBA, or possibly the DBMS, to manage such changes.) In other words, the effects of such changes must be isolated below the conceptual level, in order to preserve physical data independence.
- An *external/conceptual* mapping defines the correspondence between a particular external view and the conceptual view. In general, the differences that can exist

dump/restore purposes. In this connection, note that *multi-terabyte systems*⁶—that is, commercial database installations that store several trillions of bytes of data, loosely speaking—already exist, and systems of the future are predicted to be much larger. It goes without saying that such *VLDB* (“very large database”) systems require very careful and sophisticated administration, especially if there is a requirement for continuous availability (which there usually is). Nevertheless, we will continue to talk (for the sake of simplicity) as if there were in fact just a single database.

■ *Monitoring performance and responding to changing requirements*

As indicated in Chapter 1, the DBA is responsible for organizing the system in such a way as to get the performance that is “best for the enterprise,” and for making the appropriate adjustments—that is, tuning—as requirements change. For example, it might be necessary to reorganize the stored database from time to time to ensure that performance levels remain acceptable. As already mentioned, any change to the internal level of the system must be accompanied by a corresponding change to the definition of the conceptual/internal mapping, so that the conceptual schema can remain constant.

Of course, the foregoing is not an exhaustive list—it is merely intended to give some idea of the extent and nature of the DBA’s responsibilities.

2.8 THE DATABASE MANAGEMENT SYSTEM

The database management system (DBMS) is the software that handles all access to the database. Conceptually, what happens is the following (refer to Fig. 2.3 once again):

1. A user issues an access request, using some particular data sublanguage (typically SQL).
2. The DBMS accepts that request and analyzes it.
3. The DBMS inspects, in turn, (the object versions of) the external schema for that user, the corresponding external/conceptual mapping, the conceptual schema, the conceptual/internal mapping, and the stored database definition.
4. The DBMS executes the necessary operations on the stored database.

By way of an example, consider what is involved in the retrieval of a particular external record occurrence. In general, fields will be required from several conceptual record occurrences, and each conceptual record occurrence in turn will require fields from several stored record occurrences. Conceptually, then, the DBMS must first retrieve all required stored record occurrences, then construct the required conceptual record occur-

⁶ 1024 bytes = 1 kilobyte (KB); 1024KB = 1 megabyte (MB); 1024MB = 1 gigabyte (GB); 1024GB = 1 terabyte (TB); 1024TB = 1 petabyte (PB); 1024PB = 1 exabyte (EB or XB); 1024XB = 1 zettabyte (ZB); 1024ZB = 1 yottabyte (YB). Note in particular that a gigabyte is a billion bytes, loosely speaking (the abbreviation GB is sometimes used instead of GB). Incidentally (and contrary to popular belief), *gigabyte* is pronounced with a soft initial *g* and the *i* is long (as in *gigantic*).

rences, and then construct the required external record occurrence. At each stage, data type or other conversions might be necessary.

Of course, the foregoing description is very much simplified: in particular, it suggests that the entire process is interpretive, inasmuch as it describes the processes of analyzing the request, inspecting the various schemas, and so on, as if they were all done at run time. Interpretation, in turn, often implies poor performance, because of the run-time overhead. In practice, however, it might be possible for access requests to be *compiled* prior to run time (in particular, several of today's SQL products do this—see, for example, the annotation to references [4.13] and [4.27] in Chapter 4).

Let us now examine the functions of the DBMS in a little more detail. Those functions will include support for at least all of the following (refer to Fig. 2.4, overleaf):

■ *Data definition*

The DBMS must be able to accept data definitions (external schemas, the conceptual schema, the internal schema, and all associated mappings) in source form and convert them to the appropriate object form. In other words, the DBMS must include DDL processor or DDL compiler components for each of the various data definition languages (DDLs). The DBMS must also “understand” the DDL definitions, in the sense that, for example, it “understands” that EMPLOYEE external records include a SALARY field, and it must be able to use this knowledge in analyzing and responding to data manipulation requests (e.g., “Get employees with salary < \$50,000”).

■ *Data manipulation*

The DBMS must be able to handle requests to retrieve, update, or delete existing data in the database or to add new data to the database. In other words, the DBMS must include a DML processor or DML compiler component to deal with the data manipulation language (DML).

In general, DML requests can be planned or unplanned:

- a. A planned request is one for which the need was foreseen in advance of the time at which the request is made. The DBA will probably have tuned the physical database design in such a way as to guarantee good performance for planned requests.
- b. An unplanned request, by contrast, is an *ad hoc* query or (less likely) update—that is, a request for which the need was not seen in advance, but instead arose in a spur-of-the-moment fashion. The physical database design might or might not be well suited for the specific request under consideration.

To use the terminology introduced in Chapter 1 (Section 1.3), planned requests are characteristic of *operational* or *production* applications, while unplanned requests are characteristic of *decision support* applications. Furthermore, planned requests will typically be issued from prewritten application programs, whereas unplanned requests, by definition, will be issued interactively, typically via some *query language processor*. (In fact, as we saw in Chapter 1, the query language processor is really a built-in online application, not part of the DBMS *per se*; we include it in Fig. 2.4 only for completeness.)

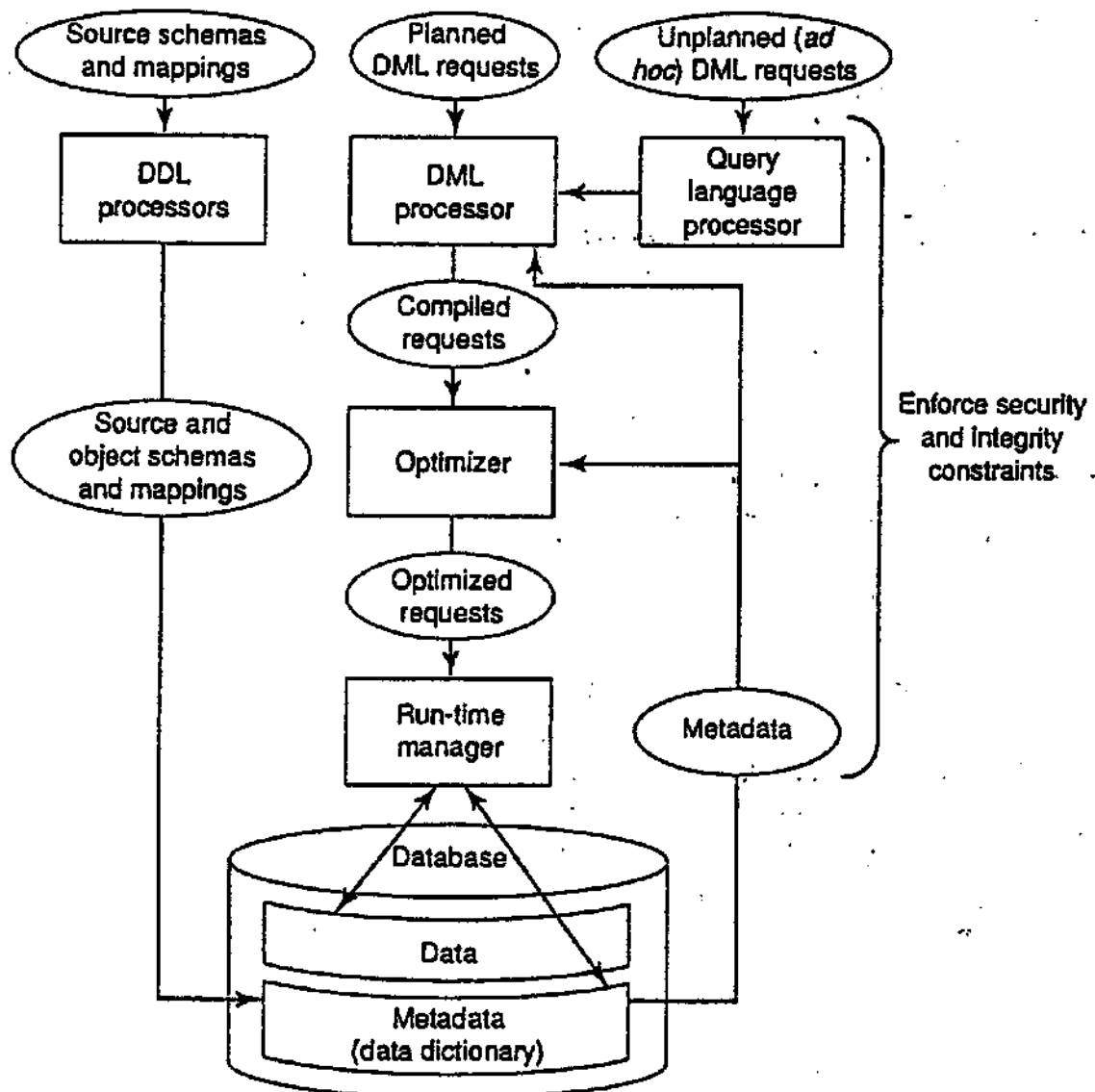


Fig. 2.4 Major DBMS functions and components

■ Optimization and execution

DML requests, planned or unplanned, must be processed by the optimizer component, whose purpose is to determine an efficient way of implementing the request.⁷ Optimization is discussed in detail in Chapter 18. The optimized requests are then executed under the control of the run-time manager. (In practice, the run-time manager will probably invoke some kind of *file* or *storage manager* to access the stored data. File managers are discussed briefly at the end of the present section.)

⁷ Throughout this book we take the term *optimization* to refer to the optimization of DML requests specifically, barring explicit statements to the contrary.

- *Data security and integrity*

The DBMS, or some subsystem invoked by the DBMS, must monitor user requests and reject any attempt to violate the security and integrity constraints defined by the DBA (see the previous section). These tasks can be carried out at compile time or run time or some mixture of the two.

- *Data recovery and concurrency*

The DBMS—or, more likely, another related software component called the transaction manager or TP monitor—must enforce certain recovery and concurrency controls. Details of these aspects of the system are beyond the scope of this chapter; see Part IV of this book for an in-depth discussion. The transaction manager is not shown in Fig. 2.4 because it is usually not part of the DBMS *per se*.

- *Data dictionary*

The DBMS must provide a data dictionary function. The data dictionary can be regarded as a database in its own right (but a system database rather than a user database); it contains “data about the data” (sometimes called metadata or descriptors)—that is, *definitions* of other objects in the system, instead of just “raw data.” In particular, all of the various schemas and mappings (external, conceptual, etc.) and all of the various security and integrity constraints will be kept, in both source and object form, in the dictionary. A comprehensive dictionary will also include much additional information, showing, for instance, which programs use which parts of the database, which users require which reports, and so on. The dictionary might even—in fact, probably should—be integrated into the database it defines and thus include its own definition. Certainly it should be possible to query the dictionary just like any other database, so that, for example, it is possible to tell which programs and/or users are likely to be affected by some proposed change to the system. See Chapter 3 for further discussion.

Note: We are touching here on an area in which there is much terminological confusion. Some people would refer to what we are calling the dictionary as a *directory* or a *catalog*—with the tacit implication that directories and catalogs are somehow inferior to a genuine dictionary—and would reserve the term *dictionary* to refer to a specific (important) kind of application development tool. Other terms that are also sometimes used to refer to this latter kind of object are *data repository* (see Chapter 14) and *data encyclopedia*.

- *Performance*

It goes without saying that the DBMS should perform all of its tasks as efficiently as possible.

We can summarize all of the foregoing by saying that the overall purpose of the DBMS is to provide the user interface to the database system. The user interface can be defined as a boundary in the system below which everything is invisible to the user. By definition, therefore, the user interface is at the *external* level. In today’s SQL products, however, there are some situations—mostly having to do with update operations—in which the external level is unlikely to differ very significantly from the relevant portion of the underlying conceptual level. We will elaborate on this issue in Chapter 10.

We conclude this section by briefly contrasting database management systems (DBMSs) with *file management systems* (*file managers* or *file servers* for short). Basically, the file manager is that component of the underlying operating system that manages stored files; loosely speaking, therefore, it is "closer to the disk" than the DBMS is. (In fact, Appendix D, online, explains how the DBMS is often built *on top of* some kind of file manager.) Thus, the user of a file management system will be able to create and destroy stored files and perform simple retrieval and update operations on stored records in such files. In contrast to the DBMS, however:

- File managers are not aware of the internal structure of stored records and hence cannot handle requests that rely on a knowledge of that structure.
- File managers typically provide little or no support for security and integrity constraints.
- File managers typically provide little or no support for recovery and concurrency controls.
- There is no genuine data dictionary concept at the file manager level.
- File managers provide much less data independence than the DBMS does.
- Files are typically not "integrated" or "shared" in the same sense that the database is, but instead are usually private to some particular user or application.

2.9 DATA COMMUNICATIONS

In this section, we briefly consider the topic of data communications. Database requests from an end user are actually transmitted from the user's computer or workstation—which might be physically remote from the database system itself—to some online application, built-in or otherwise, and thence to the DBMS, in the form of *communication messages*. Likewise, responses back from the DBMS and online application to the user's workstation are also transmitted in the form of such messages. All such message transmissions take place under the control of another software component, the data communications manager (DC manager).

The DC manager is not part of the DBMS but is an autonomous system in its own right. However, since it is clearly required to work in harmony with the DBMS, the two are sometimes regarded as equal partners in a higher-level cooperative venture called a database/data-communications system (DB/DC system), in which the DBMS looks after the database and the DC manager handles all messages to and from the DBMS, or more accurately to and from applications that use the DBMS. In this book, however, we will have comparatively little to say about message handling as such (it is a large subject in its own right). Section 2.12 does briefly discuss the question of communication *between distinct systems* (e.g., between distinct machines in a communications network such as the Internet), but that is really a separate topic.

2.10 CLIENT/SERVER ARCHITECTURE

So far in this chapter we have been discussing database systems from the point of view of the so-called ANSI/SPARC architecture. In particular, we gave a simplified picture of that architecture in Fig. 2.3. In this section we offer a slightly different perspective on the subject.

The overall purpose of a database system is to support the development and execution of database applications. From a high-level point of view, therefore, such a system can be regarded as having a very simple two-part structure, consisting of a *server*, also called the *back end*, and a set of *clients*, also called the *front ends* (refer to Fig. 2.5). *Explanation:*

1. The server is just the DBMS itself. It supports all of the basic DBMS functions discussed in Section 2.8—data definition, data manipulation, data security and integrity, and so on. In other words, “server” in this context is just another name for the DBMS.
2. The clients are the various applications that run on top of the DBMS—both user-written applications and built-in applications (i.e., applications provided by the DBMS vendor or some third party). As far as the server is concerned, of course, there is no difference between user-written and built-in applications: they all use the same interface to the server—namely, the external-level interface discussed in Section 2.3. (We note as an aside that, as mentioned in Section 2.5, certain special “utility” applications might constitute an exception to the foregoing, inasmuch as they might sometimes need to operate directly at the *internal* level of the system. Such utilities are

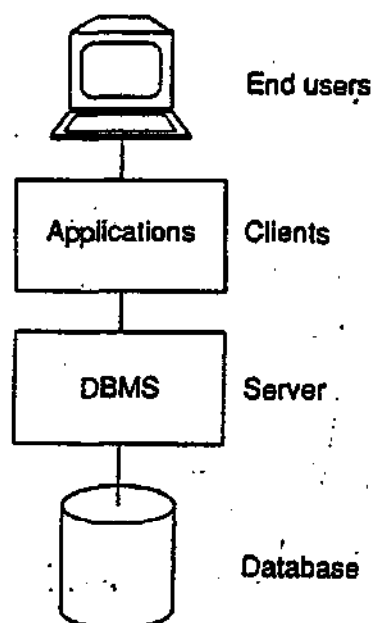


Fig. 2.5 Client/server architecture

best regarded as integral components of the DBMS, rather than as applications in the usual sense. They are discussed in more detail in the next section.)

We elaborate briefly on the question of user-written vs. vendor-provided applications:

- User-written applications are basically regular application programs, written (typically) either in a conventional 3GL such as C++ or COBOL or in some proprietary 4GL—though in both cases the language needs to be coupled somehow with an appropriate data sublanguage, as explained in Section 2.3.
- Vendor-provided applications (often called tools) are applications whose basic purpose is to assist in the creation and execution of other applications! The applications that are created are applications that are tailored to some specific task (they might not look much like applications in the conventional sense; indeed, the whole point of the tools is to allow users, especially end users, to create applications *without* having to write programs in a conventional programming language). For example, one of the vendor-provided tools will be a *report writer*, whose purpose is to allow end users to obtain formatted reports from the system on request. Any given report request can be regarded as a small application program, written in a very high-level (and special-purpose) *report writer language*.

Vendor-provided tools can be divided into several more or less distinct classes:

- a. Query language processors
- b. Report writers
- c. Business graphics subsystems
- d. Spreadsheets
- e. Natural language processors
- f. Statistical packages
- g. Copy management or "data extract" tools
- h. Application generators (including 4GL processors)
- i. Other application development tools, including computer-aided software engineering (CASE) products
- j. Data mining and visualization tools

and many others. Details of most such tools are beyond the scope of this book; however, we remark that since (as stated near the opening of this section) the whole point of a database system is to support the creation and execution of applications, the quality of the available tools is, or should be, a major factor in "the database decision" (i.e., the process of choosing the right database product). In other words, the DBMS *per se* is not the only factor that needs to be taken into account, nor even necessarily the most significant factor.

We close this section with a forward reference. Since the overall system can be so neatly divided into two parts, server and clients, the possibility arises of running the two on different machines. In other words, the potential exists for distributed processing.

Distributed processing means that distinct machines can be connected into some kind of communications network in such a way that a single data processing task can be spread across several machines in the network. In fact, so attractive is this possibility—for a variety of reasons, mainly economic—that the term *client/server* has come to apply almost exclusively to the case where client and server are indeed on different machines. We will discuss distributed processing in more detail in Section 2.12.

2.11 UTILITIES

Utilities are programs designed to help the DBA with various administration tasks. As mentioned in the previous section, some utility programs operate at the external level of the system, and thus are effectively nothing more than special-purpose applications; some might not even be provided by the DBMS vendor, but rather by some third party. Other utilities, however, operate directly at the internal level (in other words, they are really part of the server), and hence must be provided by the DBMS vendor.

Here are some examples of the kind of utilities that are typically needed in practice:

- Load routines, to create the initial version of the database from regular data files
- Unload/reload (or dump/restore) routines, to unload the database or portions thereof to backup storage and to reload data from such backup copies (of course, the "reload utility" is basically identical to the load utility just mentioned)
- Reorganization routines, to rearrange the data in the stored database for various reasons (usually having to do with performance)—for example, to cluster data in some particular way on the disk, or to reclaim space occupied by logically deleted data
- Statistical routines, to compute various performance statistics such as file sizes, value distributions, I/O counts, and so on
- Analysis routines, to analyze the statistics just mentioned

Of course, this list represents just a small sample of the range of functions that utilities typically provide; numerous other possibilities exist.

2.12 DISTRIBUTED PROCESSING

To repeat from Section 2.10, the term *distributed processing* means that distinct machines can be connected into a communications network—the Internet provides the obvious example—such that a single data processing task can span several machines in the network. (The term *parallel processing* is also used with essentially the same meaning, except that the distinct machines tend to be physically close together in a "parallel" system and need not be so in a "distributed" system; that is, they might be geographically dispersed in the latter case.) Communication among the various machines is handled by some kind of network management software, possibly an extension of the DC manager (discussed in Section 2.9), more likely a separate software component.

Many levels or varieties of distributed processing are possible. To repeat from Section 2.10, one simple case involves running the DBMS back end (the server) on one machine and the application front ends (the clients) on another. Refer to Fig. 2.6.

As mentioned at the end of Section 2.10, *client/server*, though strictly speaking a purely architectural term, has come to be almost synonymous with the arrangement illustrated in Fig. 2.6, in which client and server run on different machines. Indeed, there are many arguments in favor of such a scheme:

- The first is basically just the usual parallel processing argument: namely, two or more machines are now being applied to the overall task, and server (database) and client (application) processing are being done in parallel. Response time and throughput should thus be improved.
- Furthermore, the server machine might be a custom-built machine that is tailored to the DBMS function (a "database machine") and might thus provide better DBMS performance.
- Likewise, the client machine might be a personal workstation, tailored to the needs of the end user and thus able to provide better interfaces, high availability, faster responses, and overall improved ease of use to the user.
- Several different client machines might be able—in fact, typically will be able—to access the same server machine. Thus, a single database might be shared across several distinct clients (see Fig. 2.7).

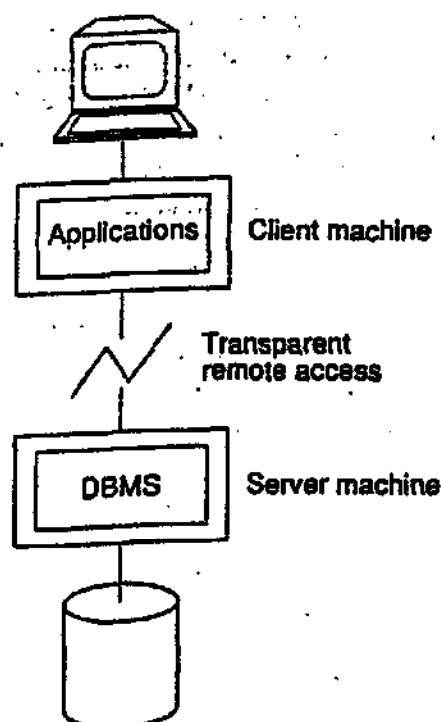


Fig. 2.6 Client(s) and server running on different machines

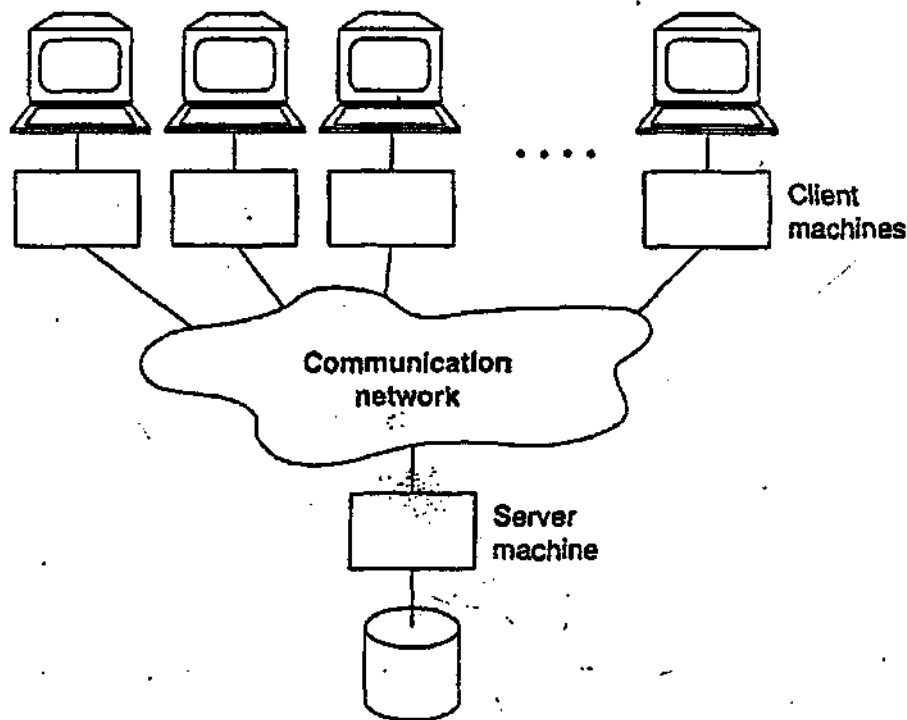


Fig. 2.7 One server machine, many client machines

In addition to the foregoing arguments, there is also the point that running the client(s) and the server on separate machines matches the way enterprises actually operate. It is quite common for a single enterprise—a bank, for example—to operate many computers, such that the data for one portion of the enterprise is stored on one computer and the data for another portion is stored on another. It is also quite common for users on one computer to need at least occasional access to data stored on another. To pursue the banking example for a moment, it is very likely that users at one branch office will occasionally need access to data stored at another. Note, therefore, that the client machines might have stored data of their own, and the server machine might have applications of its own. In general, therefore, each machine will act as a server for some users and a client for others (see Fig. 2.8); in other words, each machine will support *an entire database system*, in the sense of earlier sections of this chapter.

The final point is that a single client machine might be able to access several different server machines (the converse of the case illustrated in Fig. 2.7). This capability is desirable because, as already mentioned, enterprises do typically operate in such a manner that the totality of their data is not stored on one single machine but rather is spread across many distinct machines, and applications will sometimes need the ability to access data from more than one machine. Such access can basically be provided in two different ways:

- A given client might be able to access any number of servers, but only one at a time (i.e., each individual database request must be directed to just one server). In such a system it is not possible, within a single request, to combine data from two or more

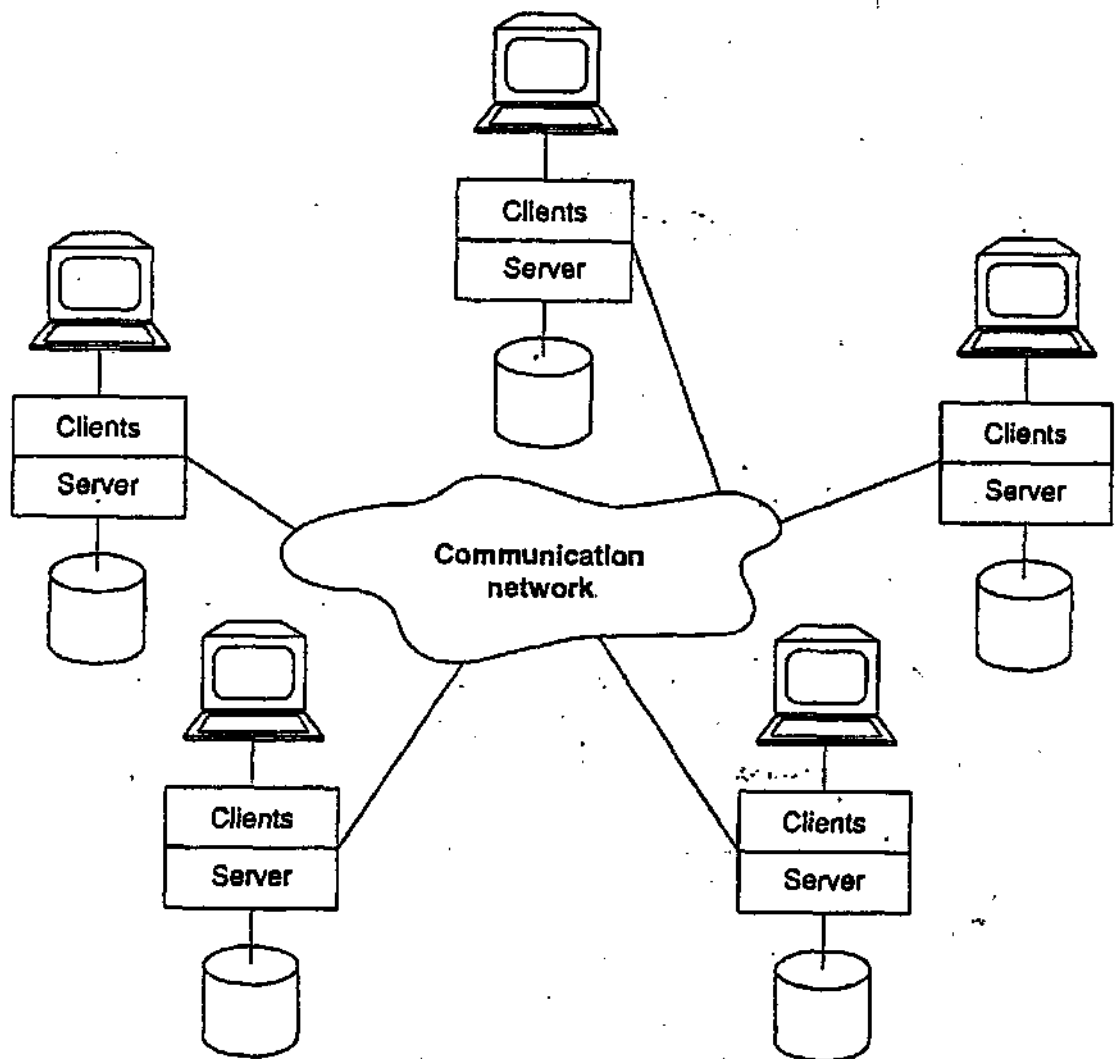


Fig. 2.8 Each machine runs both client(s) and server

different servers. Furthermore, the user in such a system has to know which particular machine holds which pieces of data.

- The client might be able to access many servers simultaneously (i.e., a single database request might be able to combine data from several servers). In this case, the servers look to the client from a logical point of view as if they were really a single server, and the user does not have to know which machines hold which pieces of data.

This latter case constitutes what is usually called a distributed database system. Distributed database is a big topic in its own right; carried to its logical conclusion, full distributed database support implies that a single application should be able to operate "transparently" on data that is spread across a variety of different databases, managed by a

variety of different DBMSs, running on a variety of different machines, supported by a variety of different operating systems, and connected by a variety of different communication networks—where “transparently” means the application operates from a logical point of view as if the data were all managed by a single DBMS running on a single machine. Such a capability might sound like a pretty tall order, but it is highly desirable from a practical perspective, and much effort has been devoted to making such systems a reality. We will discuss such systems in detail in Chapter 21.

2.13 SUMMARY

In this chapter we have looked at database systems from an architectural point of view. First, we described the ANSI/SPARC architecture, which divides a database system into three levels, as follows: The internal level is the one closest to physical storage (i.e., it is the one concerned with the way the data is stored); the external level is the one closest to the users (i.e., it is the one concerned with the way the data is viewed by individual users); and the conceptual level is a level of indirection between the other two (it provides a *community view* of the data). The data as perceived at each level is described by a schema (or several schemas, in the case of the external level). Mappings define the correspondence between (a) a given external schema and the conceptual schema, and (b) the conceptual schema and the internal schema. Those mappings are the key to the provision of logical and physical data independence, respectively.

Users—that is, end users and application programmers, both of whom operate at the external level—interact with the data by means of a data sublanguage, which consists of at least two components, a data definition language (DDL) and a data manipulation language (DML). The data sublanguage is embedded in a host language. Please note, however, that the boundaries (a) between the host language and the data sublanguage and (b) between the DDL and the DML are primarily conceptual in nature; ideally they should be “transparent to the user.”

We also took a closer look at the functions of the DBA and the DBMS. Among other things, the DBA is responsible for creating the internal schema (physical database design); by contrast, creating the conceptual schema (logical or conceptual database design) is the responsibility of the *data administrator*. And the DBMS is responsible, among other things, for implementing DDL and DML requests from the user. The DBMS is also responsible for providing some kind of data dictionary function.

Database systems can also be conveniently thought of as consisting of a server (the DBMS itself) and a set of clients (the applications). Client and server can and often will run on distinct machines, thus providing one simple kind of distributed processing. In general, each server can serve many clients, and each client can access many servers. If the system provides total “transparency”—meaning that each client can behave as if it were dealing with a single server on a single machine, regardless of the overall physical state of affairs—then we have a genuine distributed database system.

EXERCISES

2.1 Draw a diagram of the database system architecture presented in this chapter (the ANSI/SPARC architecture).

2.2 Explain the following in your own words:

back end	front end
client	host language
conceptual DDL, schema, view	load
conceptual/internal mapping	logical database design
data definition language	internal DDL, schema, view
data dictionary	physical database design
data manipulation language	planned request
data sublanguage	reorganization
DB/DC system	server
DC manager	stored database definition
distributed database	unload/reload
distributed processing	unplanned request
external DDL, schema, view	user interface
external/conceptual mapping	utility

2.3 Describe the sequence of steps involved in retrieving a particular external record occurrence.

2.4 List the major functions performed by the DBMS.

2.5 Distinguish between logical and physical data independence.

2.6 What do you understand by the term *metadata*?

2.7 List the major functions performed by the DBA.

2.8 Distinguish between the DBMS and a file management system.

2.9 Give some examples of vendor-provided tools.

2.10 Give some examples of database utilities.

2.11 Examine any database system that might be available to you. Try to map that system to the ANSI/SPARC architecture as described in this chapter. Does it cleanly support the three levels of the architecture? How are the mappings between levels defined? What do the various DDLs (external, conceptual, internal) look like? What data sublanguage(s) does the system support? What host languages? Who performs the DBA function? Are there any security or integrity facilities? Is there a dictionary? Is it self-describing? What vendor-provided applications does the system support? What utilities? Is there a separate DC manager? Are there any distributed processing capabilities?

REFERENCES AND BIBLIOGRAPHY

Most of the following references are fairly old by now, but they are still relevant to the concepts introduced in the present chapter. See also the references in Chapter 14.

2.1 ANSI/X3/SPARC Study Group on Data Base Management Systems: Interim Report, *FDT* (bulletin of ACM SIGMOD) 7, No. 2 (1975).